Daniel Casasfranco
Juan Jose Castro
Alejandra de la Bastida 202520379

Entrega 2

a. **Un diagrama de clases de diseño que incluya todas las clases, incluyendo sus relaciones, atributos y métodos.**

Descripción del diagrama de clases de diseño

El diagrama se organiza en torno a Club, que actúa como fachada: es la única clase que invocan los programas de consola y guarda las colecciones del sistema. En el diagrama de diseño estas aparecen como atributos de Club; en el diagrama de alto nivel aparecen como asociaciones. Las clases se agrupan así:

- **Usuarios**: Usuario es abstracta y se especializa en Socio, Empleado, la cual tiene Entrenador y Fisioterapeuta, y Administrador. Un Entrenador puede ser también Socio
- **Deportes de conjunto:** Equipo agrupa Jugadores. Partido se especializa en PartidoEquipo, que tiene PartidoInterno y PartidoCompeticion, y en PartidoPractica. Las estadísticas de cada deporte heredan de EstadisticaPartido. Las Sanciones se asocian a un jugador y, si es correcto, a una competencia
- **Competencias**: Competencia se especializa en ASCUN que tiene Fases y TorneoDeLosCerros. InscripcionEquipo registra la participación de un equipo, y JugadorNomina su nómina oficial
- **Deportes individuales y entrenamiento:** InscripcionDisciplina guarda el nivel y el ELO de un socio, con su historial en RegistroRanking. Instalación se especializa en cinco tipos de cancha y mesa. Reserva y SesionEntrenamiento usan instalaciones, y Convocatoria registra la asistencia de cada convocado
- **Tienda y cafetería:** Producto se especializa en ArticuloTienda, Bebida y Snack. El inventario se maneja con ExistenciaInventario y UbicacionInventario. Venta se especializa en VentaTienda y VentaCafeteria, y se compone de LineaVenta
- **Turnos**: Turno y SolicitudCambioTurno
- **Reglas**: ReglaDeporte guarda mínimos de jugadores, puntos por resultado y máximo de instalaciones simultáneas de cada deporte

Los atributos que no tiene simbolo son privados, los marcados con # son protected. Solo se muestran los constructores de las clases creadas en los diagramas de secuencia, se omiten getters y setters.

Decisiones clave y su justificación

- **Club como fachada:** Los programas de consola solo conocen a Club, que coordina cada escenario. Así la consola no depende de las clases del dominio
- **Las reglas viven en el objeto que tiene la información:** Equipo valida su cupo y su categoría, Instalacion su capacidad y disponibilidad, e InscripcionDisciplina el nivel de sus jugadores. Club solo pregunta y decide qué hacer con la respuesta. Esto evita que Club crezca con la lógica de todas las clases.

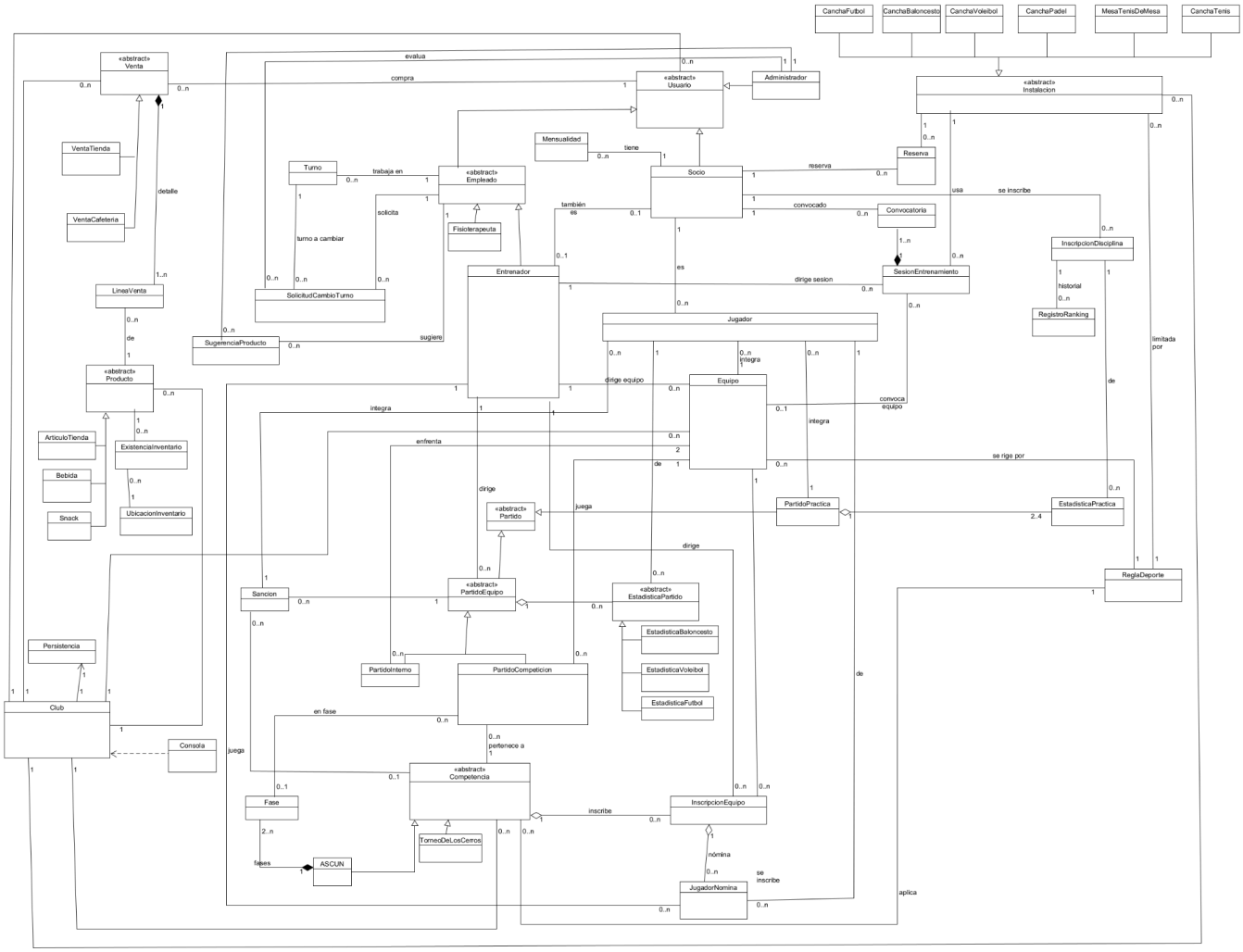
- **Sancion como clase y no como un booleano**: Al principio el jugador tenía un atributo suspendido. Lo reemplazamos por objetos Sancion con tipo, partidos de suspensión, partidos cumplidos y estado, porque un booleano no distingue en qué competencia se aplica la sanción ni cuándo se termina. Si competencia es null, la sanción viene de un partido interno
- **Polimorfismo:** en vez de condicionales por tipo Producto.permitido() lo sobrescribe Bebida para cumplir con la regla de una bebida caliente no se permite en una instalación techada, Competencia.acepta() es común a ASCUN y TorneoDeLosCerros, y Venta.calcularTotal() lo implementa distinto cada tipo de venta
- **Estadísticas como Map<String,Integer>**: Una sola firma sirve para fútbol, baloncesto, voleibol y los deportes individuales
- **Persistencia aislada en una clase única:** Club invoca persistencia solo después de validar. El resto del dominio no conoce los archivos

Todas las clases del dominio implementan Serializable, y Persistencia escribe y lee el objeto Club completo

Cómo funciona:

- guardar(club) escribe el Club completo en club.dat. Se llama después de cada operación exitosa, como muestran los diagramas de secuencia.
- cargar() lee club.dat y devuelve el Club. Si el archivo no existe, devuelve un Club vacío, con lo que la primera ejecución arranca sin datos.
- Contenido. Como Club contiene todas las listas, el único archivo incluye usuarios, con login y password, equipos, jugadores, competencias, instalaciones, reservas, sesiones, partidos, ventas, productos, inventario, reglas y solicitudes.

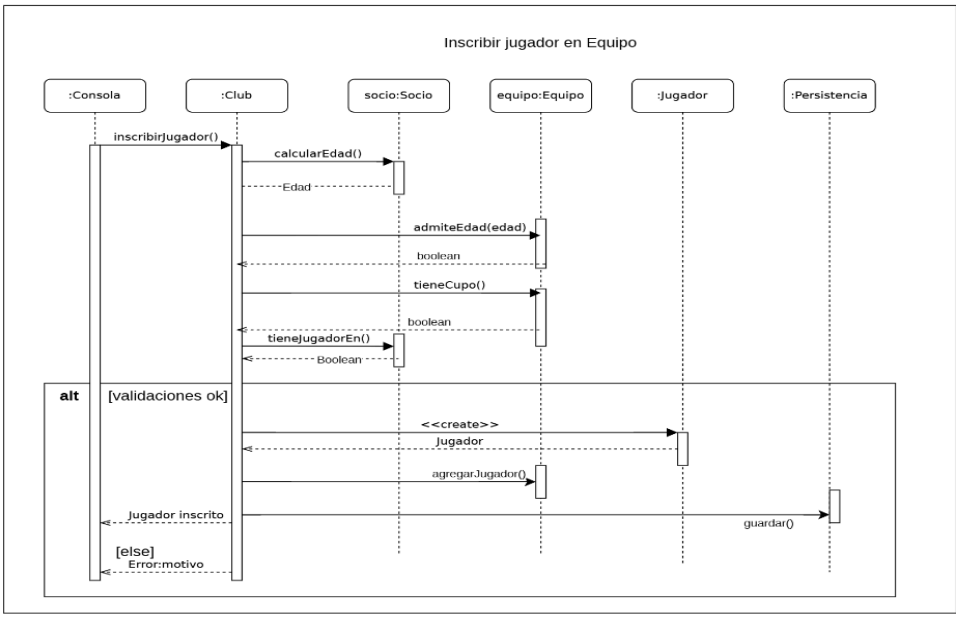
b. Un diagrama de clases de alto nivel, que incluya todas las clases y sus relaciones, pero no todos los métodos ni atributos.



c. Diagramas de secuencia para las funcionalidades que usted considere críticas

En todos los diagramas de secuencia hay persistencia y sólo se invoca después de qué pasan las validaciones. Los casos de rechazo se muestran en el bloque alt con su rama con su rama, él sí else

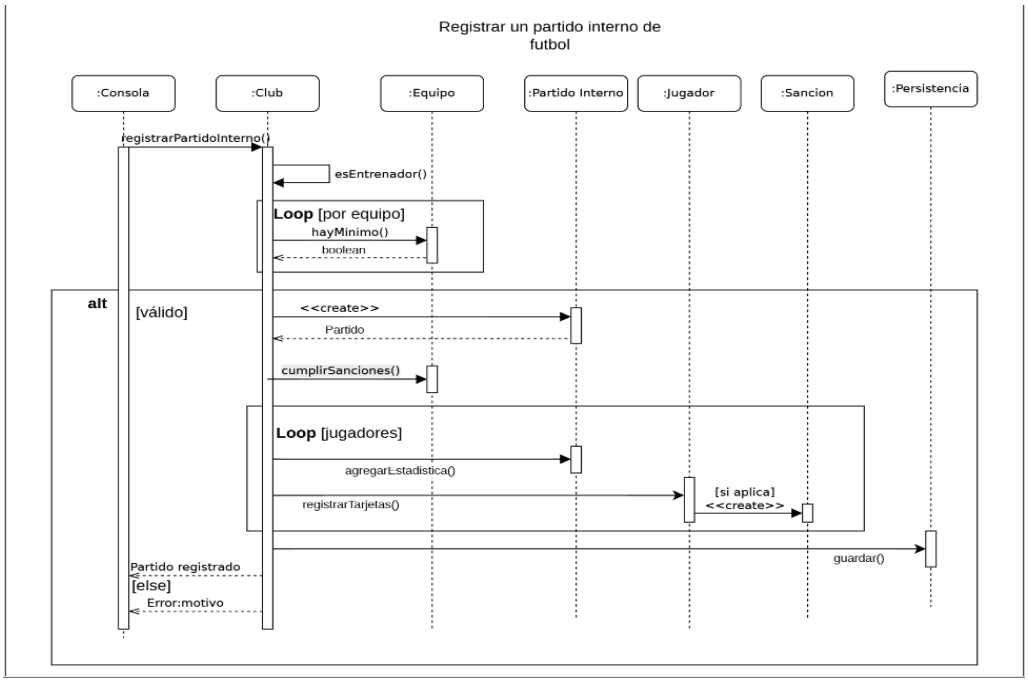
1. Inscribir jugador en Equipo



Justificación

En este diagrama, Club obtiene la edad del Socio, consulta al Equipo si admite esa edad y si tiene cupo, además, consulta al Socio si ya tiene un jugador en ese deporte. En caso de que las tres validaciones se cumplan, crea el jugador, lo agrega al equipo y guarda. El jugador solo se crea después de validar, así que nunca queda una inscripción inválida

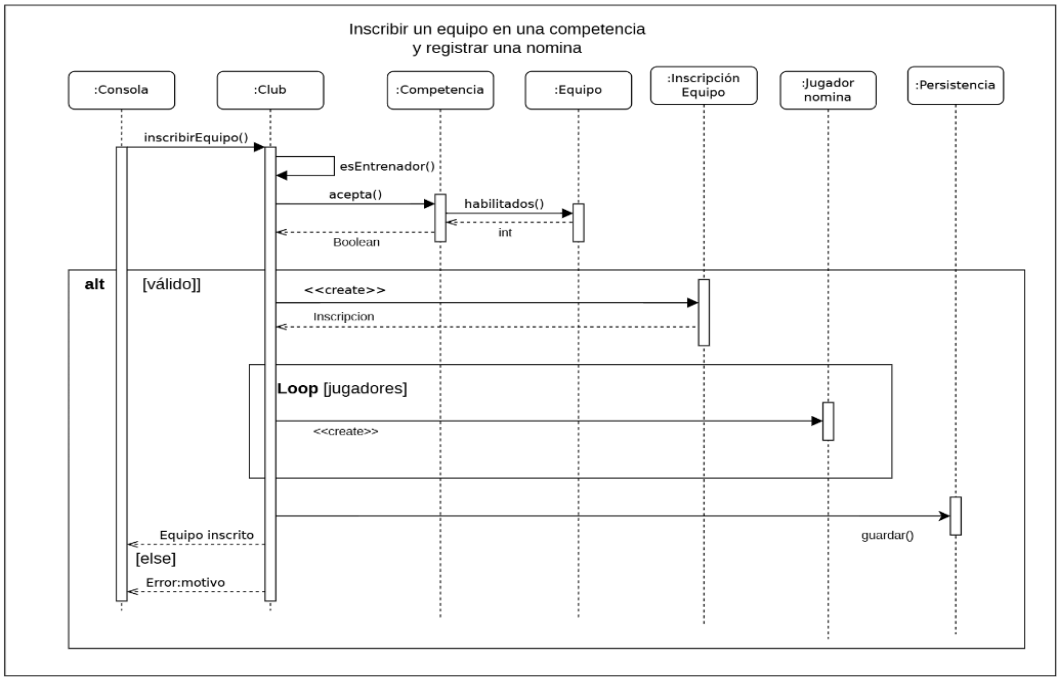
2. Registrar un partido interno de futbol



Justificación

Un entrenador registra el resultado. Club Verifica el rol y que pues que pasen el mínimo de jugadores habilitados. Después que el partido, agrega la estadística de cada jugador y sus tarjetas, así crea una Sanción Cuándo corresponde. Las funciones son objetos con estado y vigencia y no un booleano en el jugador. CumplirSanciones() Va antes de las tarjetas nuevas para que una sancion recién creada no se cuenta como cumplida en el partido que el origino

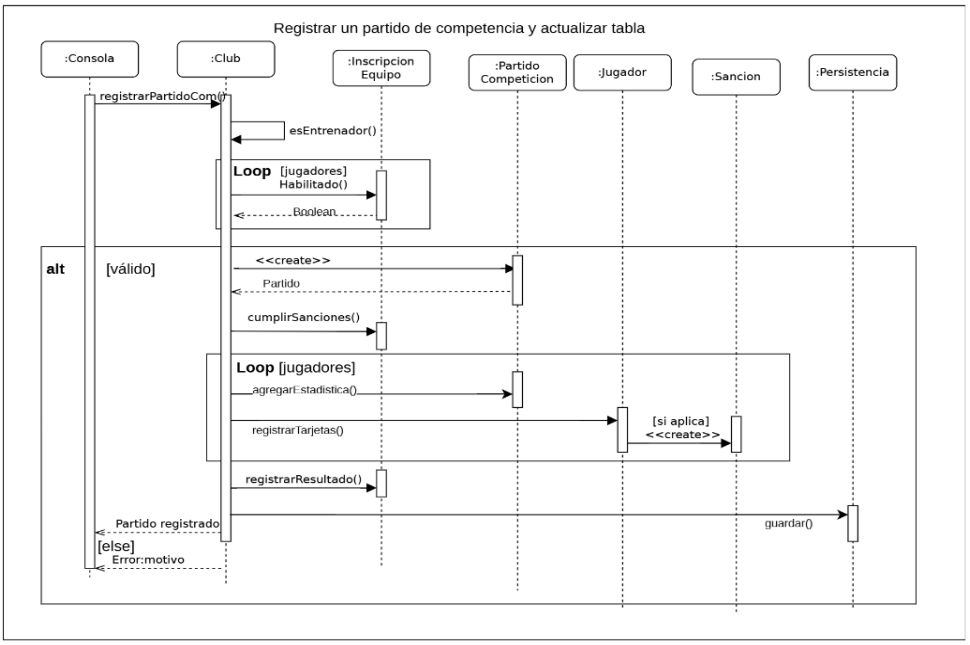
3. Inscribir un equipo en una competencia y registrar una nómina



Justificación

Club verifica que quien llama sea entrenador, porque esa función no la tienen todos los empleados. Competencia.acepta() Válida, la categoría, el deporte y la nómina mínima consultando al Equipo cuántos jugadores habilitados tiene. Si es válido, si crean la InscripciónEquipo y un jugadorNomina por jugador. Acepta() esta es la clase abstracta Competencia esta es la clase abstracta, competencia para que ASCUN y TorneoDeLosCerros lo vuelvan a usar, y la nómina se modela con objetos separados para llevar las sanciones por competencia

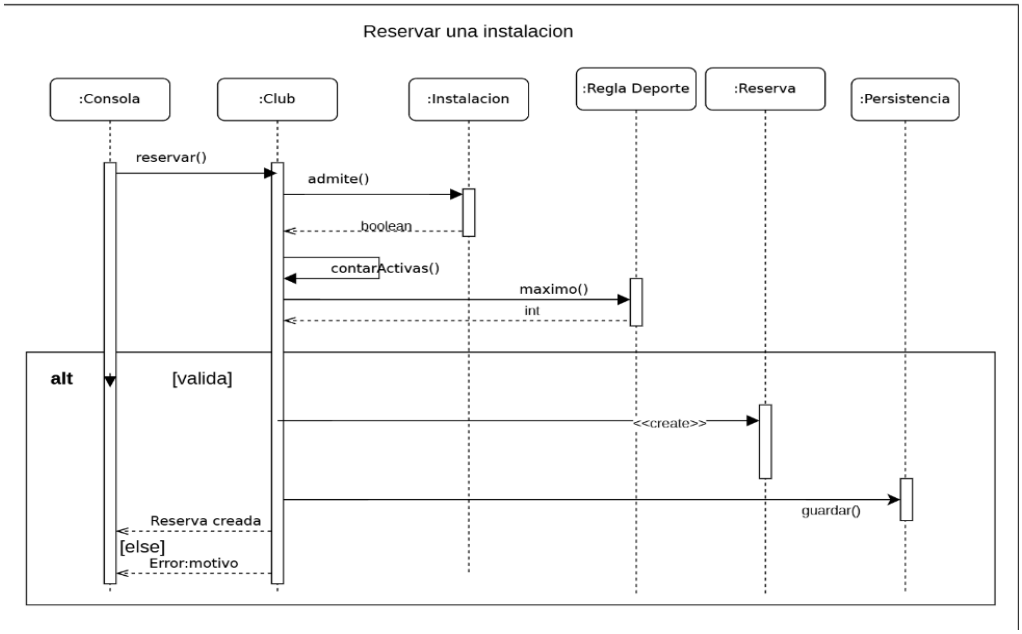
4. Registrar un partido de competencia y actualizar tabla



Justificación

El entrenador registra el partido. InscripciónEquipo.habilitado() tiene dos reglas, el jugador debe ser parte de la nómina y no tener una sanción activa en esa competencia. Después de crear el partido, van las sanciones, que se registran, estadísticas y tarjetas, y se actualiza la inscripción con el resultado. La tabla de posiciones se actualiza en InscripcionEquipo con puntos, ganados, empatados y perdidos según la regla de deporte

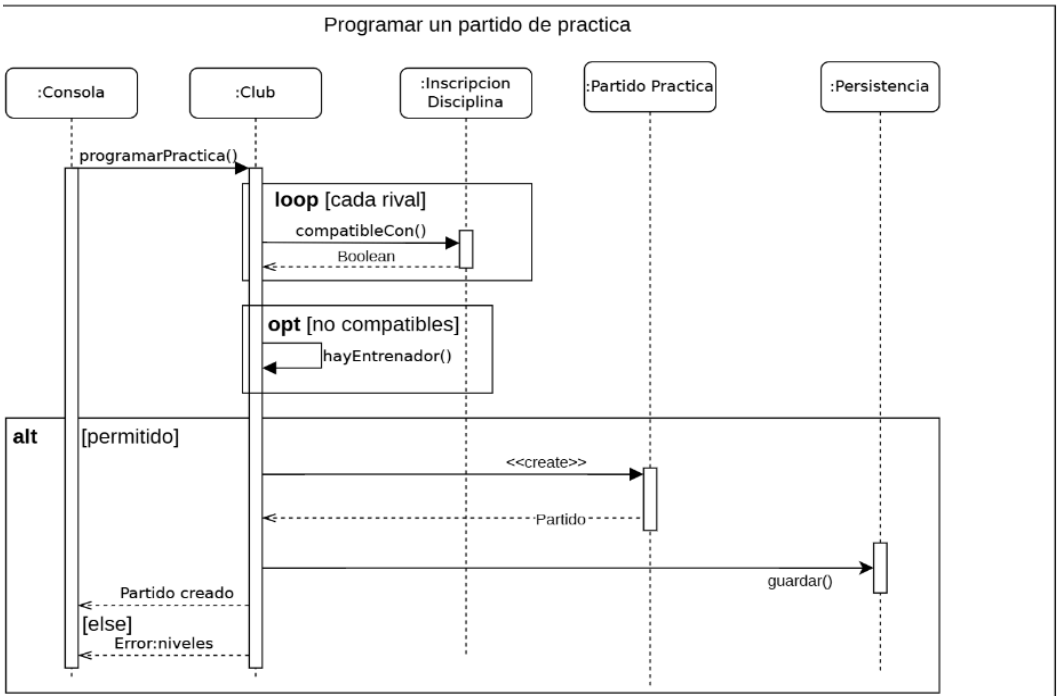
5. Reservar una instalación



Justificación

Instalacion.admite() valida capacidad y modalidad, como por ejemplo en el padel que solo admite dobles. Club cuenta las reservas activas del deporte en ese horario y las compara con regla deporte. EN caso de que haya cupo crea la Reserva y la guarda. Solo un socio puede reservar

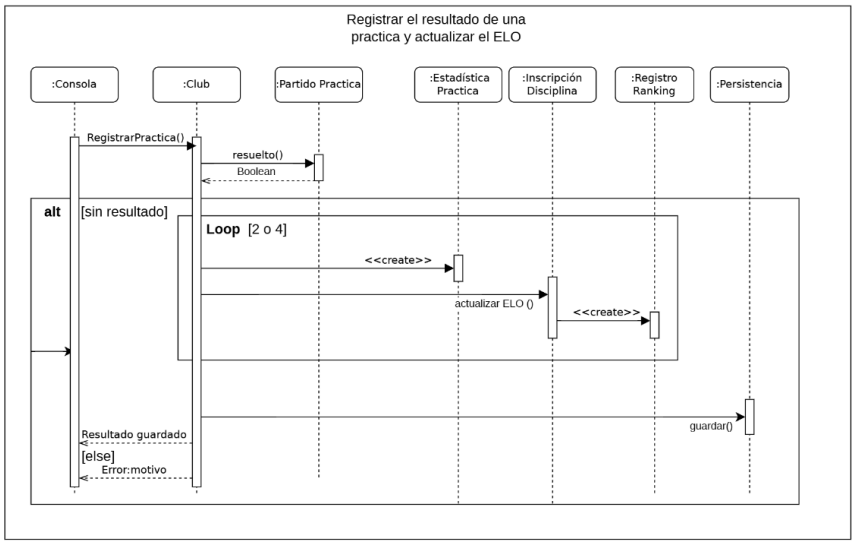
6. Programar un partido de práctica



Justificación

Para cada rival InscripcionDisciplina.CompatibleCon() se encarga de comparar niveles, si no son compatibles se consulta si algún participante tambien es entrenador. SI se cumple se crea el PartidoPractica con su modalidad. La consulta de si es entrenador va en un bloque opt, para que solo se ejecute cuando es necesario. Y el loop por rival permite que sea para modalidad sencillos y dobles

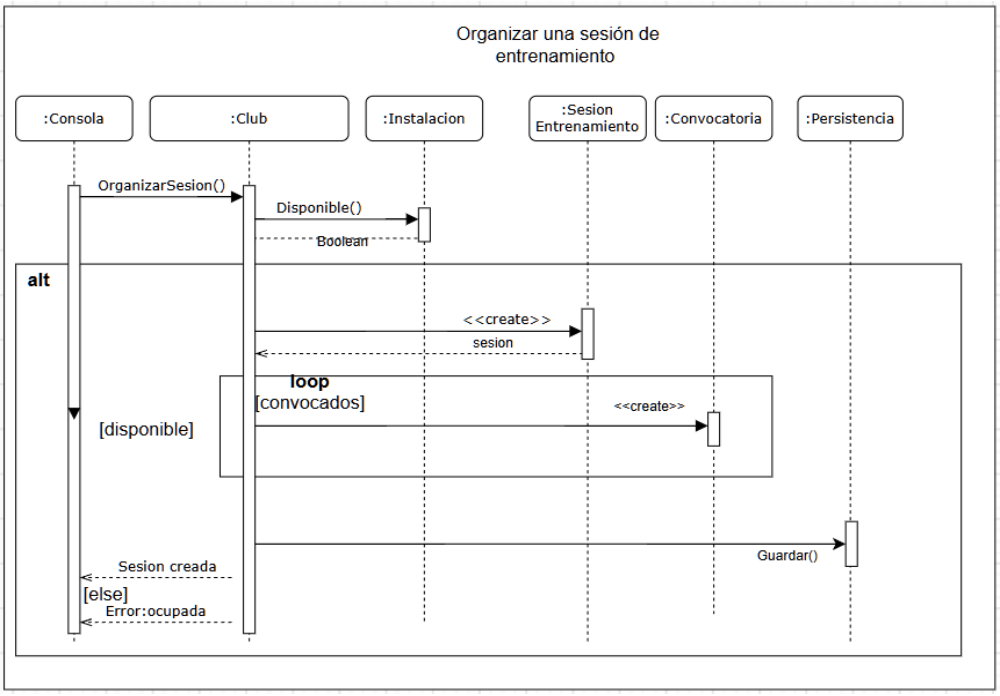
7. Registrar el resultado de una practica y actualizar el ELO



Justificación

Resuelto() evita duplicados en el momento de registrar el resultado de un partido. Por cada participante se crea EstadisticaPractica con su lado, y la inscripcion calcula su ELO y deja un RegistroRanking con el puntaje anterior, el nuevo y el motivo. Cada inscripción calcula su correspondiente ELO, y el historial en RegistroRanking llena los reportes de rankings. En dobles se usa el promedio del ELO de los rivales

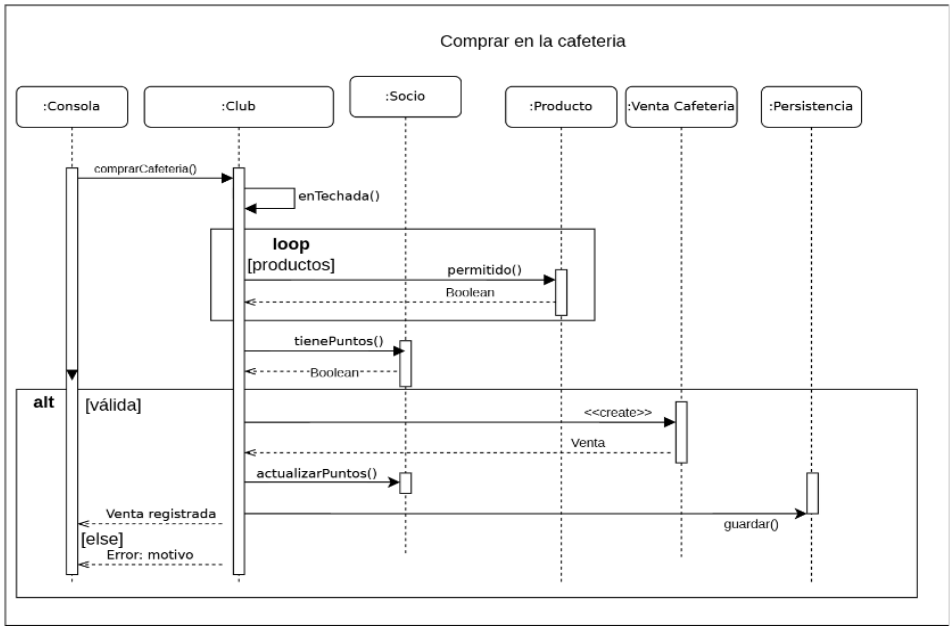
8. Organizar una sesión de entrenamiento



Justificación

Instalacion.Disponible() mira si el horario esta libre, considera reservas como otras sesiones. Si esta libre se crea la SesionEntrenamiento y una Convocatoria por cada convocado. Las convocatorias se crean sin asistencia, que se completa al terminar la sesión

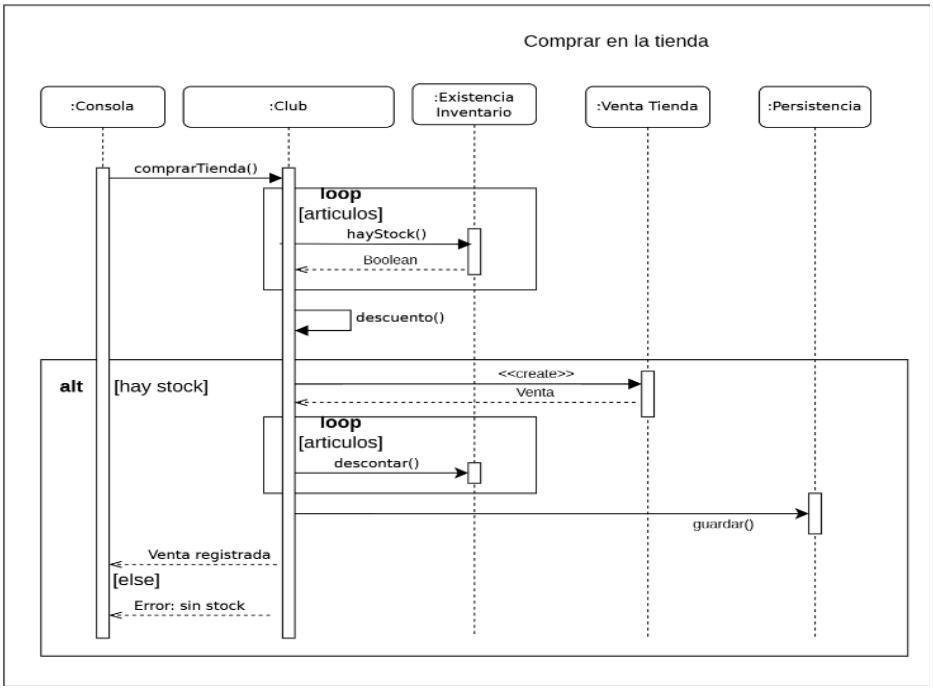
9. Comprar en la cafetería



Justificación

Club determina si el socio esta en una instalación techada, revisa si cada producto esta permitido y verifica que el socio tenga los puntos que quiera usar. Si todo es valido crea la venta, actualiza los puntos y guarda. La restricción de bebidas calientes se resuelven con polimorfismo, Bebida sobrescribe Permitido(). VentaCafeteria calcula impuesto, propina, total y puntos ganados

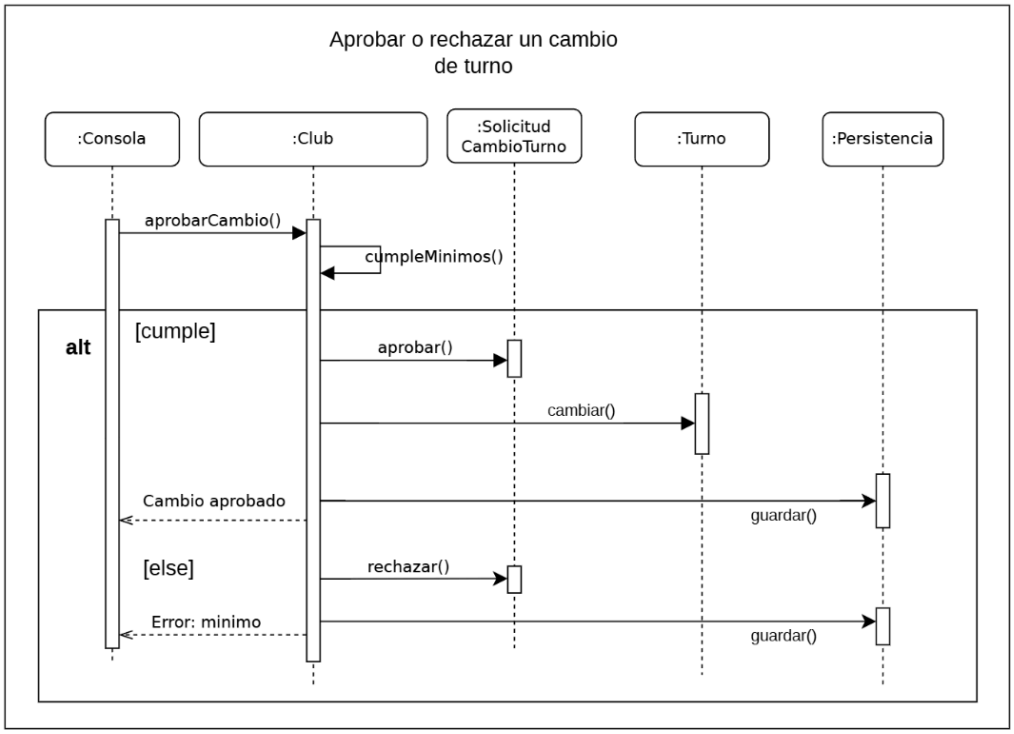
10. Comprar en la tienda



Justificación

Se valida el stock de todos los artículos, si alcanza se calcula el descuento, se crea la VentaTienda se descuenta del inventario y se guarda. Hay dos loops, validar todo antes de modificar evita dejar el inventario a medias. El descuento se calcula en Club porque debe identificar de quién es el codigo, es decir, 15% si es del propio empleado, 8% si es compartido y VentaTienda aplica el IVA del 19%

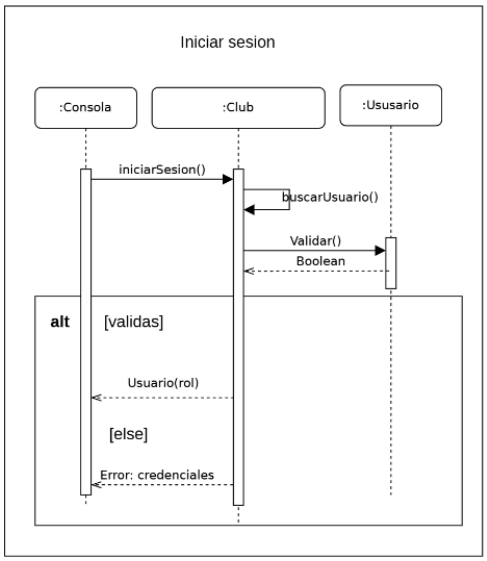
11. Aprobar o rechazar un cambio de turno



Justificación

Club calcula si el cambio de turno mantiene la regla de dos entrenadores y un fisioterapeuta disponible. Si se cumple, la solicitud se aprueba y el turno se cambia, si no se rechaza, en los dos casos se guarda. CumpleMiimos() esta en CLub porque necesita contar a todos los empleados. Se guarda en las dos ramas porque rechazar tambien cambia el estado de la solicitud y debe quedar en el historial

12. Iniciar sesión



Justificación

Club busca al usuario por Loquin y le pide validar la contraseña. Si es válido, devuelve el usuario con su rol, sino, da un error. Usuario.Validar() guarda la credencial, de modo que él Password no sale del objeto. El login inexistente y el password incorrecto devuelven el mismo error para no revelar qué dato falló, y el rol del usuario devuelto determina las funciones disponibles.